Construction of Knowledge Graphs: State and Challenges

Marvin Hofer [b,*], Daniel Obraczka [b], Alieh Saeedi [a,b], Hanna Köpcke [c] and Erhard Rahm [a,b]

[a] *Dept. of Computer Science, Leipzig University, Germany*

[b] *Center for Scalable Data Analytics and Artificial Intelligence (ScaDS.AI) Dresden/Leipzig, Germany*

[c] *Faculty Applied Computer Sciences and Biosciences, Mittweida University of Applied Sciences, Germany*

arXiv:2302.11509v2 [cs.AI] 11 Oct 2023

Abstract. With knowledge graphs (KGs) at the center of numerous applications such as recommender systems and question answering, the need for generalized pipelines to construct and continuously update such KGs is increasing. While the individual steps that are necessary to create KGs from unstructured (e.g. text) and structured data sources (e.g. databases) are mostly well-researched for their one-shot execution, their adoption for incremental KG updates and the interplay of the individual steps have hardly been investigated in a systematic manner so far. In this work, we first discuss the main graph models for KGs and introduce the major requirements for future KG construction pipelines. Next, we provide an overview of the necessary steps to build high-quality KGs, including cross-cutting topics such as metadata management, ontology development, and quality assurance. We then evaluate the state of the art of KG construction w.r.t the introduced requirements for specific popular KGs as well as some recent tools and strategies for KG construction. Finally, we identify areas in need of further research and improvement.

Keywords:

1. Introduction

Aggregated machine-readable information in the form of knowledge graphs (KG) serves as the backbone of numerous data science applications nowadays, ranging from question-answering [1] over recommendation systems [2] to predicting drug-target interactions [3]. The ever-changing nature of information necessitates the design of KG construction pipelines that are able to incorporate new information continuously. In the development of such a system, knowledge engineering teams have to deal with a variety of challenges from tackling scalability and heterogeneous data sources to tracking the provenance of data. Given the usually large volume of data that needs to be integrated, such pipelines have to be automatized as much as possible while aiming at a high degree of data quality.

Knowledge graphs generally integrate heterogeneous data from a variety of sources with unstructured and semi-structured data of different modalities (e.g., pictures, audio, text) as well as structured data such as databases or other KGs in a semantically rich way. The construction of a KG therefore encompasses a multi-disciplinary effort requiring expertise from research areas such as natural language processing (NLP), data integration, knowledge representation, and knowledge management. Knowledge graphs are at the center of numerous use-cases for data analysis and decision support. In the clinical setting, enriching patient data with medical background knowledge enables improved clinical decision support [4]. Sonntag et. al. [5] argue that properly aligning the semantic labels attached to the patient data with medical ontologies is crucial in creating meaningful access to the heterogeneous patient data. Knowledge graphs are also used to organize the relevant information for fast emerging global topics,

*Corresponding author. E-mail: hofer@informatik.uni-leipzig.de.

such as pandemics (e.g., Covid-19) or natural disasters [6]. Machine learning also benefits from KGs as a source of labeled training data or other input data [7, 8] thereby supporting the development of knowledge- and data-driven AI approaches [9]. KGs can further be combined with Large Language Models (LLMs) to improve factual correctness and explanations in question-answering, e.g. with ChatGPT, thereby promoting quality and interpretability of AI decision-making [10, 11].

Improving the pipelines that build such knowledge graphs and enabling them to efficiently keep current and semantically meaningful aggregated knowledge is therefore an effort that benefits a wide range of application areas. However, KGs are currently created often in a batch-like manner so that the respective pipelines are unfit to incorporate new incoming facts into a KG without full re-computation of the individual tasks. Furthermore, different steps of the pipelines often require manual intervention, thereby limiting scalability to large data volumes and increasing the time for updating a KG.

There is a growing number of surveys about knowledge graphs, especially on their general characteristics and usage forms [7–9]. An excellent tutorial-style overview about the construction and curation of KGs is provided in [12] with a focus on integrating data from textual and semi-structured data sources such as Wikipedia. Other surveys focus on KG construction with specific technologies [13, 14] or only a single domain such as for geographical data [15]. We discuss related surveys on KG construction in Section 6 and contrast them with our approach.

This survey article provides a concise yet comprehensive entry into the current state of the art in KG construction for readers new to the topic, as well as contributing valuable guidance for researchers, engineers, and experts by highlighting existing solution approaches, tools, and identifying open gaps in the areas. We first outline the main requirements for the construction and continuous maintenance of KGs distilled from the literature as well as our experience and reasoning. Next, we give an overview of the concrete subtasks of KG construction and current solution approaches. Furthermore, we select 23 KG-specific construction approaches as well as generic toolsets based on the criteria discussed in Section 4.1 and evaluate and compare them w.r.t. the requirements introduced in Section 2.3. Finally, we identify open challenges and current limitations and thus areas for further research.

Our survey builds on previous studies for KG construction but differs in essential aspects as will be explained in detail in Section 6. In contrast with most other surveys, we explicitly specify the main requirements for KG construction and use these as a guideline for evaluating current solutions and identifying open challenges. We are also more comprehensive in several important aspects as we cover different graph data models (RDF and property graphs) and deal with incremental KG construction and data integration including incremental entity resolution in much more detail. We also provide a comparison between many carefully selected KG-specific construction approaches and toolsets and identify open challenges that go beyond those discussed in previous surveys.

The further structure of this survey is as follows:

- After a definition of KGs and a comparative discussion of graph data models for KGs we introduce and categorize the general requirements for incremental KG construction in Section 2.
- In Section 3 we provide an overview of the main tasks in incremental KG construction pipelines and proposed solution approaches for them.
- We then investigate and compare existing construction efforts for selected KGs as well as within recent tools for KG construction w.r.t. the requirements introduced earlier. This also allows us to identify tasks that are not yet supported well.
- Section 5 discusses open challenges for KG construction.
- Section 6 contains a more in-depth comparison with the related work.
- Finally, we offer concluding remarks and a summary.

2. KG background and requirements for KG construction

We first outline the notion of knowledge graph (KG) used in this paper which is based on the integration of information from multiple sources. We then briefly introduce and compare the two most popular graph data models for KGs, namely RDF and property graphs. Finally, we outline the main requirements or desiderata for largely automatic construction and maintenance of KGs.

2.1. Knowledge Graph

KGs typically realize physical data integration, where the information from different sources is combined in a new graph-like representation[1]. KGs are schema-flexible and can thus easily accommodate and interlink heterogeneously structured entities. This is in contrast to the use of data warehouses as a popular approach for physical data integration. Data warehouses focus on integrating data within a structured (relational) database with a relatively static schema optimized for certain multi-dimensional data analysis. Schema evolution is a manual and tedious process making it difficult to add new data sources or new kinds of information not conforming to the schema. KGs are less restricted and can better deal with heterogeneous information derived from semi- and unstructured data from potentially many sources.

Although the term *knowledge graph* goes back as far as 1973 [18], it gained popularity through the 2012 blog post[2] about the Google KG. Afterward, several related definitions of knowledge graphs were proposed, either in research papers [8, 12, 19–21] or by companies using or supporting KGs (OpenLink, Ontotext, Neo4J, TopQuadrant, Amazon, Diffbot[3], Google). Ehrlinger et al. [20] give a comprehensive overview of KG definitions and provide their own: "A knowledge graph acquires and integrates information into an ontology and applies a reasoner to derive new knowledge." Hogan et al. [22] argue that this definition is very specific and excludes various industrial KGs which helped to popularize the concept.

We therefore define KGs more inclusively as a graph of data consisting of semantically described entities and relations of different types that are integrated from different sources. Entities have unique identifier. KG entities and relations can be semantically described by an ontology [23]. A KG's ontology defines the concepts, relationships, and rules governing the *semantic structure* within a KG of one or several domains that also include the types and properties of entities and their relationships. To structure data in a KG, common ontology relationships euch as *is-a* and *has-a* are used to represent taxonomic hierarchies and possessive relations between entities. Furthermore, an ontology can enable the inference of new implicit knowledge from the explicitly represented information in the KG [21].

Figure 1 visualizes a simplified KG example with integrated information from several domains where ontological information such as types or is-a relations are dashed. There are ten entities of the following eight types: Country (*Ireland*), City (*Limerick*), Artist (*Aphex Twin*), Album (*Selected Ambient Works 85-9*), Record Label (*R & S*), Genre (*Techno*, *Ambient Techno*), Song (*Xtal*, *Ageispolis*) and Year (*1992*). Ontological *is-a* (sub-class) relations interrelate City and Country with Place, Artist with Person, Album with Music Release Type, and Record Label with Organisation. The domain is further described by the named relationships: country, birthPlace, artist, label, writtenBy, yearReleased, founded, broader, genre, yearProduced, partOf. Based on the given relationships and typing, further information is inferable, e.g., Aphex Twin's broader birthplace is Ireland, the song Xtal is also of genre Techno, Aphex Twin being of the type Artist means this instance is also of the type Person (for readability, not all possible inferences are denoted).

2.2. Graph Models

To represent and use KGs as informally defined above, a powerful graph data model is needed that supports entities and relations of different types as well as their ontological description and organization [24]. Moreover, the graph data model should provide a comprehensive query language and possibly more advanced graph analysis or mining capabilities, e.g., for clustering of similar entities or determining graph embeddings for machine learning tasks. Support for integrity constraints is also desirable to automatically control the consistency and therefore, quality of graph data to some extent. Furthermore, it should be possible to represent annotating metadata of KG entities, e.g., about their origin and transformation during KG construction. It is also desirable to reflect the development of the KG over time so that a temporal KG analysis is supported. This can be achieved by a temporal graph data

[1] While we will focus on the predominant physical data integration of KGs, there are also some virtual data integration approaches, e.g., to keep data source more autonomous [16, 17].

[2] https://blog.google/products/search/introducing-knowledge-graph-things-not/

[3] https://blog.diffbot.com/knowledge-graph-glossary/

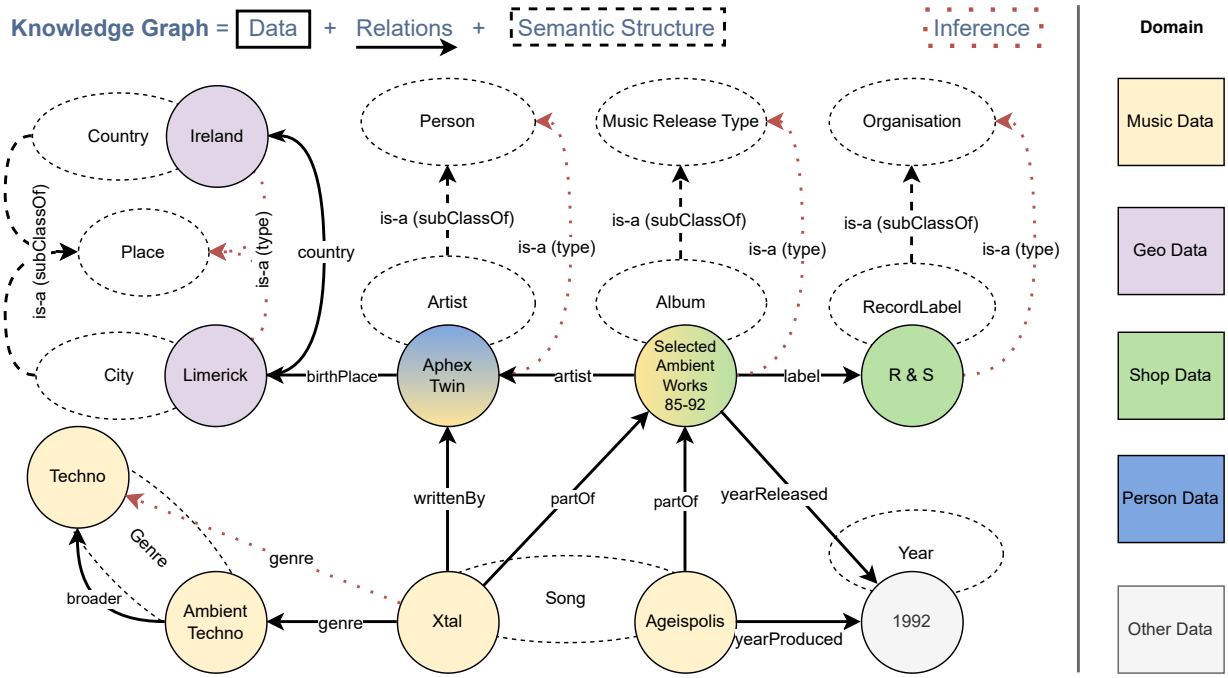

Fig. 1. Simplified Knowledge Graph (KG) example demonstrating integrated information from five domains, showcasing ten entities of eight types connected by twelve relationships (two distinct is-a relations). Dashed lines indicate semantic structures (ontology or graph schema) such as entity types. Inferences can be made based on the relationships and typing, revealing additional information such as the broader birthplace of Aphex Twin being Ireland and Xtal belonging to the Techno genre (Not all possible inferences are shown for clarity).

Table 1
Comparison of RDF and property graphs.

	Resource Description Framework (RDF)	Property Graph Model (PGM)
base constructs	triples <subject, predicate, object>	labeled vertices and edges and their properties
entity identity	IRI-based	local (implementation-specific)
node classification	rdf:type triples	type labels
ontology support	RDFS, OWL2 vocabularies	limited, e.g., schema graph
integrity constraints	SHACL, SHEX	PG-Keys, PG-Schema
query language	SPARQL(-Star)	Cypher, Gremlin, G-Core, PGQL
exchange format	N-Triples, N-Quads, (RDF/XML, JSONLD)	application specific e.g., PGEF, GDL
meta information	reification, singleton-property, (RDF-Star)	dedicated properties

model with time metadata for every entity and relation and temporal query possibilities, e.g., to determine previous states of the KG or to find out what has been changed in a certain time interval. The temporal development of a KG might alternatively be reflected with a versioning concept where new KG versions are periodically released. Finally, the graph data model should facilitate the KG construction process and its different tasks for acquiring, transforming and integrating heterogeneous data from different sources. This can be supported by suitable formats to seamlessly exchange data of the chosen graph model between different steps and processing nodes of a KG construction pipeline.

The most common graph models used for KGs are the Resource Description Framework and the Property Graph Model. In the following, we briefly describe both and discuss how they meet the introduced desiderata. Table 2.2 summarizes some of the key differences of both models. At the end, we also contrast the different terminology of the models and specify the terms used in the rest of this paper.

Resource Description Framework (RDF) is a framework or data model to present data in a graph-like fashion, and was initially developed to describe metadata of web-resources (namely the Semantic Web) [25]. Today, the W3C proposes many technologies around RDF that help build and use knowledge graphs either as part of the Linked Data Cloud or in an encapsulated environment. KGs are represented by a set of <subject, predicate, object> triples that uniformly represent named relations (predicates) of entities (subjects) to either attribute values (literals) or other entities (objects). Entities are usually assigned an IRI (Internationalized Resource Identifier) that can refer to either a global or local namespace. In addition to IRIs, entities can identify by a blank node identifier that is only unique inside an RDF dataset. Sets of triples may also be grouped within named graphs to aggregate more information by extending the triple structure to quads of the form <subject, predicate, object, named-graph>. Standard RDF does not support edge properties, although the RDF-Star extension[4] includes a similar feature, where single triples are usable in the subject or object part of another triple.

The RDF standard also defines vocabularies such as RDF Schema (RDFS) to further express semantic structure by allowing the definition of classes, properties and their hierarchies. An RDF resource's type (or entity class) is assigned by using the standard RDF vocabulary[5] to define triples of the form `<s rdf:type o>` where o is the class (type) of the resource. In addition to RDFS, a widely used approach for defining ontologies is the Web Ontology Language (OWL), more specifically, the current version OWL 2, which adds semantics to the data using a variety of axioms.

Besides syntax validation (triples/quads, URIs, datatypes) RDF triple stores do not provide a standard method to define and validate graph data integrity or shape constraints (similar to relation database schemata). Therefore overlaid solutions such as SHACL (Shape Constraint Language [26]) or ShEx (Shape Expressions [27]) are developed that can be used to validate the semantic correctness of the graphs structure, node or property constraints, cardinalities, and other constructs.

While some RDF stores or triple stores are built from scratch to optimize the management of RDF triples, others might use existing SQL or NoSQL systems in the underlying database processing layer. The primary query language for RDF (moreover, the Semantic Web) is the standardized language SPARQL[6], with an extended version for RDF-Star called SPARQL-Star. Standard exchange formats for RDF are N-Triples, N-Quads, Turtle, or adapted syntax formats like RDF/XML and JSON-LD.

There are different possibilities to assign metadata to entities, relations and properties like using RDF-Star or named graphs as explained and evaluated in [28, 29]. The usage of support constructs for metadata management generally increases the complexity of the graph structure and queries and can possibly increase processing time.

There is some work around the representation, querying storage and other aspects of temporal information in RDF [30]. The investigated methods focus on different temporal granularity and dimensions, including approaches that target to query single snapshots, time windows or to inspect the evolution of temporal graphs.

As many knowledge graphs are in RDF, several frameworks have been developed to perform graph analytics, algorithms, or mining tasks using RDF as input [31].

Property Graph Model (PGM) [32]. The property graph data model, also called Labeled Property Graph (LPG), supports the flexible definition of graph structures with heterogeneous nodes (vertices) and directed edges to represent entities of different kinds, and the relationships between them. Both nodes and edges can have multiple (type-)labels expressing their role in the modeled domain, e.g., *User* as a node label and *follows* as edge label. Additionally, properties (in the form of key-value pairs) can be assigned to both nodes and edges. Further, in the most common implementations, vertices and edges are specified by a unique identifier. While label and property names represent some schema-like information, there is intentionally no predefined schema to allow flexible incorporation of heterogeneous entities and relations of different kinds (although a schema graph can be inferred from the type information [33]). There is no built-in support for ontologies, e.g., to provide is-a relations between entity categories. Embedded metadata can be relatively easy maintained for entities and relationships by using dedicated properties, e.g. for provenance or time annotations.

[4]https://www.w3.org/2022/08/rdf-star-wg-charter/
[5]http://www.w3.org/1999/02/22-rdf-syntax-ns#
[6]https://www.w3.org/TR/sparql11-overview/